

■ MÓDULO DE PAGOS Y RECORDATORIOS

Roadmap Completo de Desarrollo

Fecha: 14/04/2026

Proyecto: Sistema Modular React + Vite + Supabase

Autor: Abraham Kohan

■ ÍNDICE

- 1. Análisis de la Base de Datos Actual
- 2. Recomendación Técnica
- 3. Arquitectura del Proyecto
- 4. Roadmap de Desarrollo (7 Fases)
- 5. Mejoras sobre Notion
- 6. Guía UI/UX Minimalista
- 7. Integración con CRM Existente
- 8. Uso Standalone (Independiente)
- 9. Estimación de Tiempos
- 10. Prompt para Claude Code

1. ■ ANÁLISIS DE LA BASE DE DATOS ACTUAL

Estructura actual en Notion - Base de datos 'Gastos Abraham':

Campo	Tipo	Opciones/Formato
Nombre	Título (text)	Texto libre
Número	Number	Formato: dólar (\$)
Categoría	Select	Servicios, Expensas, Honorarios, Impuestos, Trámites, Otros
Responsable	Select	Spirit Mariscal, Nora, Orestes, Abraham
Fecha Vencimiento	Date	Fecha simple (sin hora)
Texto	Text	Notas adicionales

Colores de categorías en Notion:

- Servicios: Azul
- Expensas: Verde
- Honorarios: Naranja
- Impuestos: Púrpura
- Trámites: Naranja
- Otros: Gris

2. ■ RECOMENDACIÓN TÉCNICA

Stack Tecnológico Recomendado

- **React 19** - Framework principal
- **Vite** - Build tool (consistente con tu CRM)
- **TypeScript** - Tipado estático
- **Tailwind CSS 4** - Estilos (ya lo usás)
- **Supabase** - Base de datos, auth, notificaciones
- **React Query (TanStack Query)** - Estado del servidor
- **date-fns** o **day.js** - Manejo de fechas
- **React Hook Form** - Formularios
- **Zod** - Validación de schemas

Estrategia de Implementación: GIT SUBMODULE (Recomendado)

■ **Opción recomendada: Git submodule que se puede integrar en cualquier proyecto.**

■ Ventajas del Git Submodule:

- Lo podés insertar en tu CRM inmobiliario
- Lo podés reutilizar en otros proyectos futuros
- Mantiene su propia base de datos pero se integra fácilmente
- No necesitás publicar a npm ni mantener versiones
- Actualizaciones centralizadas (cambios en un lugar, afectan todos los proyectos)

Otras opciones disponibles:

Opción B: Paquete npm privado (@abrahamkohan/payments-module)

→ Más profesional pero requiere versionado y publicación

Opción C: Microservicio standalone con API REST

→ Para usar en múltiples apps no-React (más complejo)

3. ■■ ARQUITECTURA DEL PROYECTO

Estructura de Carpetas

```
payments-reminders/
■■■ src/
■■■ components/
■■■ PaymentsList/
■■■ PaymentsList.tsx
■■■ PaymentCard.tsx
■■■ PaymentFilters.tsx
■■■ PaymentSort.tsx
■■■ PaymentForm/
■■■ PaymentForm.tsx
■■■ CategorySelect.tsx
■■■ DatePicker.tsx
■■■ EmojiPicker.tsx
■■■ ColorPicker.tsx
■■■ PaymentViews/
■■■ TableView.tsx
■■■ KanbanView.tsx
■■■ CalendarView.tsx
■■■ Dashboard/
■■■ PaymentsDashboard.tsx
■■■ StatsCard.tsx
■■■ CategoryChart.tsx
■■■ shared/
■■■ CategoryBadge.tsx
■■■ DueDateBadge.tsx
■■■ IconSelector.tsx
■■■ hooks/
■■■ usePayments.ts
■■■ useReminders.ts
■■■ useNotifications.ts
■■■ useRecurrence.ts
■■■ lib/
■■■ supabase.ts
■■■ date-utils.ts
■■■ currency-utils.ts
■■■ types/
■■■ payment.types.ts
■■■ reminder.types.ts
■■■ config/
■■■ categories.ts
■■■ icons.ts
■■■ colors.ts
■■■ index.tsx (exporta PaymentsModule)
■■■ supabase/
■■■ migrations/
■■■ 001_create_payments_table.sql
■■■ 002_create_reminders_table.sql
■■■ functions/
■■■ check-reminders/
■■■ index.ts
■■■ package.json
■■■ vite.config.ts
■■■ tsconfig.json
■■■ README.md
```

Schema de Base de Datos (Supabase/PostgreSQL)

```
-- Tabla principal de pagos
CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
```

```

-- Campos básicos
name TEXT NOT NULL,
amount DECIMAL(10,2),
category TEXT CHECK (category IN (
    'servicios', 'expensas', 'honorarios',
    'impuestos', 'tramites', 'otros'
)),
responsable TEXT,
due_date DATE,
notes TEXT,

-- Personalización
icon TEXT DEFAULT '■',
color TEXT,

-- Estado
is_paid BOOLEAN DEFAULT FALSE,
paid_at TIMESTAMPTZ,

-- Recurrencia
recurrence TEXT CHECK (recurrence IN (
    'none', 'daily', 'weekly', 'monthly', 'yearly'
)) DEFAULT 'none',
parent_payment_id UUID REFERENCES payments(id),

-- Multi-tenant (para múltiples usuarios)
user_id UUID REFERENCES auth.users(id),

CONSTRAINT valid_amount CHECK (amount >= 0)
);

-- Tabla de recordatorios
CREATE TABLE payment_reminders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    payment_id UUID REFERENCES payments(id) ON DELETE CASCADE,
    reminder_date TIMESTAMPTZ NOT NULL,
    days_before INTEGER NOT NULL,
    is_sent BOOLEAN DEFAULT FALSE,
    sent_at TIMESTAMPTZ,
    channel TEXT CHECK (channel IN ('email', 'push', 'sms'))
);

-- Índices para optimización
CREATE INDEX idx_payments_due_date ON payments(due_date);
CREATE INDEX idx_payments_user_id ON payments(user_id);
CREATE INDEX idx_payments_category ON payments(category);
CREATE INDEX idx_payments_responsible ON payments(responsible);
CREATE INDEX idx_payments_is_paid ON payments(is_paid);
CREATE INDEX idx_reminders_date ON payment_reminders(reminder_date);
CREATE INDEX idx_reminders_is_sent ON payment_reminders(is_sent);

-- Trigger para updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_payments_updated_at
BEFORE UPDATE ON payments
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

-- RLS (Row Level Security) para multi-tenant
ALTER TABLE payments ENABLE ROW LEVEL SECURITY;
ALTER TABLE payment_reminders ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view own payments"
ON payments FOR SELECT
USING (auth.uid() = user_id);

CREATE POLICY "Users can insert own payments"
ON payments FOR INSERT
WITH CHECK (auth.uid() = user_id);

```

```
CREATE POLICY "Users can update own payments"  
  ON payments FOR UPDATE  
  USING (auth.uid() = user_id);  
  
CREATE POLICY "Users can delete own payments"  
  ON payments FOR DELETE  
  USING (auth.uid() = user_id);
```

4. ■■■ ROADMAP DE DESARROLLO

El desarrollo se divide en 7 fases principales. Cada fase es incremental y entregable.

FASE 1: Setup & Arquitectura Base (1-2 días)

Objetivos:

- Crear estructura del proyecto
- Configurar Vite + React + TypeScript + Tailwind
- Setup de Supabase (cliente, tablas, migraciones)
- Definir tipos TypeScript base

Tareas específicas:

1. Inicializar proyecto con Vite

```
npm create vite@latest payments-reminders -- --template react-ts
cd payments-reminders
npm install
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

2. Instalar dependencias principales

```
npm install @supabase/supabase-js
npm install @tanstack/react-query
npm install react-hook-form @hookform/resolvers zod
npm install date-fns
npm install lucide-react # iconos
npm install @radix-ui/react-select @radix-ui/react-popover # UI components
```

3. Crear archivo de tipos base (types/payment.types.ts)

```
export type Category =
  | 'servicios'
  | 'expensas'
  | 'honorarios'
  | 'impuestos'
  | 'tramites'
  | 'otros';

export type Recurrence =
  | 'none'
  | 'daily'
  | 'weekly'
  | 'monthly'
  | 'yearly';

export interface Payment {
  id: string;
  created_at: string;
  updated_at: string;
  name: string;
  amount: number | null;
  category: Category;
  responsible: string;
  due_date: string | null;
  notes: string | null;
  icon: string;
  color: string | null;
  is_paid: boolean;
  paid_at: string | null;
  recurrence: Recurrence;
}
```



```

    parent_payment_id: string | null;
    user_id: string;
  }

export interface PaymentReminder {
  id: string;
  payment_id: string;
  reminder_date: string;
  days_before: number;
  is_sent: boolean;
  sent_at: string | null;
  channel: 'email' | 'push' | 'sms';
}

```

4. Configurar Supabase client (lib/supabase.ts)

```

import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

export const supabase = createClient(supabaseUrl, supabaseKey);

```

5. Crear migraciones SQL en Supabase

→ Ejecutar el SQL del schema mostrado anteriormente

■ Entregable Fase 1:

- Proyecto inicializado y funcionando
- Base de datos creada en Supabase
- Tipos TypeScript definidos

FASE 2: Componentes Base & CRUD (2-3 días)

Objetivos:

- Crear componentes visuales básicos
- Implementar CRUD de pagos
- Hook usePayments para React Query

Componentes a crear:

1. **PaymentCard.tsx** - Card individual de pago
 - Muestra: nombre, monto, categoría badge, fecha
 - Checkbox para marcar como pagado
 - Botones: editar, duplicar, eliminar
2. **PaymentForm.tsx** - Formulario crear/editar
 - Campos: nombre, monto, categoría, responsable, fecha, notas
 - Emoji picker para ícono
 - Color picker opcional
 - Validación con Zod + React Hook Form
3. **CategoryBadge.tsx** - Badge de categoría
 - Muestra ícono + nombre
 - Colores según categoría
4. **DueDateBadge.tsx** - Badge de fecha
 - Color según proximidad:
 - Rojo: vencido
 - Naranja: vence en 1-3 días
 - Amarillo: vence en 4-7 días
 - Gris: más de 7 días

Hook principal (hooks/usePayments.ts):

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { supabase } from '@lib/supabase';
import type { Payment } from '@types/payment.types';

export function usePayments() {
  const queryClient = useQueryClient();

  // Query: obtener todos los pagos
  const { data: payments, isLoading } = useQuery({
    queryKey: ['payments'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('payments')
        .select('*')
        .order('due_date', { ascending: true });

      if (error) throw error;
      return data as Payment[];
    },
  });
}
```

```

// Mutation: crear pago
const createPayment = useMutation({
  mutationFn: async (newPayment: Omit<Payment, 'id' | 'created_at' | 'updated_at'>) => {
    const { data, error } = await supabase
      .from('payments')
      .insert([newPayment])
      .select()
      .single();

    if (error) throw error;
    return data;
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['payments'] });
  },
});

// Mutation: actualizar pago
const updatePayment = useMutation({
  mutationFn: async ({ id, ...updates }: Partial<Payment> & { id: string }) => {
    const { data, error } = await supabase
      .from('payments')
      .update(updates)
      .eq('id', id)
      .select()
      .single();

    if (error) throw error;
    return data;
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['payments'] });
  },
});

// Mutation: eliminar pago
const deletePayment = useMutation({
  mutationFn: async (id: string) => {
    const { error } = await supabase
      .from('payments')
      .delete()
      .eq('id', id);

    if (error) throw error;
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['payments'] });
  },
});

// Mutation: marcar como pagado
const togglePaid = useMutation({
  mutationFn: async ({ id, is_paid }: { id: string; is_paid: boolean }) => {
    const { error } = await supabase
      .from('payments')
      .update({
        is_paid,
        paid_at: is_paid ? new Date().toISOString() : null
      })
      .eq('id', id);

    if (error) throw error;
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['payments'] });
  },
});

return {
  payments,
  isLoading,
  createPayment,
  updatePayment,
  deletePayment,
  togglePaid,
};

```

}

■ Entregable Fase 2:

- CRUD completo funcionando
- Poder crear, editar, eliminar pagos
- Vista lista básica de pagos

FASE 3: Vistas y Filtros (2-3 días)

Objetivos:

- Implementar vistas: Tabla, Kanban, Calendario
- Sistema de filtros avanzado
- Ordenamiento flexible

Vistas a implementar:

1. Vista Tabla (TableView.tsx)

- Similar a Notion
- Columnas: Nombre, Monto, Categoría, Responsable, Vencimiento, Estado
- Click en fila para editar
- Ordenamiento por columna

2. Vista Kanban (KanbanView.tsx)

- Columnas por categoría
- Drag & drop para cambiar categoría
- Contador de pagos por columna
- Suma total por columna

3. Vista Calendario (CalendarView.tsx)

- Calendario mensual
- Pagos agrupados por fecha de vencimiento
- Click en día para ver detalles
- Indicadores visuales: vencidos (rojo), próximos (naranja)

Sistema de Filtros (PaymentFilters.tsx):

- Filtrar por categoría (multi-select)
- Filtrar por responsable (multi-select)
- Filtrar por estado: Todos / Pendientes / Pagados
- Filtrar por rango de fechas
- Búsqueda por texto en nombre/notas

Ordenamiento (PaymentSort.tsx):

- Por fecha de vencimiento (asc/desc)
- Por monto (asc/desc)
- Por nombre (A-Z / Z-A)
- Por fecha de creación

■ Entregable Fase 3:

- 3 vistas funcionales (Tabla, Kanban, Calendario)

- Sistema completo de filtros
- Ordenamiento por múltiples criterios

FASE 4: Sistema de Recordatorios (2-3 días)

Objetivos:

- Configurar recordatorios automáticos
- Edge Function para chequeo diario
- Envío de emails con Supabase

Funcionalidades:

1. Configuración de recordatorios por pago

- Al crear/editar pago: opción de agregar recordatorios
- Ej: 'Recordarme 7 días antes', '3 días antes', '1 día antes'
- Múltiples recordatorios por pago

2. Edge Function en Supabase (cron job diario)

```
// supabase/functions/check-reminders/index.ts
import { serve } from 'https://deno.land/std@0.168.0/http/server.ts';
import { createClient } from '@supabase/supabase-js';

serve(async (req) => {
  const supabase = createClient(
    Deno.env.get('SUPABASE_URL') ?? '',
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') ?? ''
  );

  // 1. Buscar pagos pendientes con vencimiento próximo
  const today = new Date();
  const sevenDaysFromNow = new Date();
  sevenDaysFromNow.setDate(today.getDate() + 7);

  const { data: payments } = await supabase
    .from('payments')
    .select('*')
    .eq('is_paid', false)
    .gte('due_date', today.toISOString())
    .lte('due_date', sevenDaysFromNow.toISOString());

  // 2. Para cada pago, crear recordatorios si no existen
  for (const payment of payments || []) {
    const dueDate = new Date(payment.due_date);
    const daysUntilDue = Math.ceil(
      (dueDate.getTime() - today.getTime()) / (1000 * 60 * 60 * 24)
    );

    // Crear recordatorios predefinidos
    const reminderDays = [7, 3, 1]; // días antes

    for (const days of reminderDays) {
      if (daysUntilDue === days) {
        // Verificar si ya existe
        const { data: existing } = await supabase
          .from('payment_reminders')
          .select('id')
          .eq('payment_id', payment.id)
          .eq('days_before', days)
          .single();

        if (!existing) {
          // Crear recordatorio
          await supabase
            .from('payment_reminders')
            .insert({
              payment_id: payment.id,
              reminder_date: new Date().toISOString(),
              days_before: days,
              channel: 'email',
            });
        }
      }
    }
  }
});
```

```

    });

    // Enviar email
    await fetch('https://api.resend.com/emails', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${Deno.env.get('RESEND_API_KEY')}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        from: 'Recordatorios <pagos@tudominio.com>',
        to: payment.user_email,
        subject: `Recordatorio: ${payment.name} vence en ${days} día(s)`,
        html: `
          <h2>Recordatorio de Pago</h2>
          <p><strong>${payment.name}</strong> vence en <strong>${days} día(s)</strong>.</p>
          <p>Monto: ${payment.amount ? '$' + payment.amount : 'No especificado'}</p>
          <p>Fecha de vencimiento: ${new Date(payment.due_date).toLocaleDateString()}</p>
        `
      })
    });

    // Marcar como enviado
    await supabase
      .from('payment_reminders')
      .update({ is_sent: true, sent_at: new Date().toISOString() })
      .eq('payment_id', payment.id)
      .eq('days_before', days);
  }
}

return new Response(
  JSON.stringify({ success: true, processed: payments?.length || 0 }),
  { headers: { 'Content-Type': 'application/json' } }
);
});

```

3. Configurar Cron Job en Supabase

- Ejecutar función todos los días a las 8:00 AM
- Usar pg_cron extension de PostgreSQL

```

-- En Supabase SQL Editor
SELECT cron.schedule(
  'check-payment-reminders',
  '0 8 * * *', -- Todos los días a las 8 AM
  $$
  SELECT net.http_post(
    url := 'https://YOUR_PROJECT.supabase.co/functions/v1/check-reminders',
    headers := '{"Authorization": "Bearer YOUR_ANON_KEY"}'::jsonb
  );
  $$
);

```

■ Entregable Fase 4:

- Sistema de recordatorios configurado
- Edge function funcionando
- Emails automáticos enviándose

FASE 5: Pagos Recurrentes (2 días)

Objetivos:

- Permitir configurar pagos que se repiten
- Auto-crear siguiente pago al marcar como pagado
- Gestionar series de pagos

Funcionalidades:

1. Campo 'Recurrencia' en formulario

- Opciones: Ninguna, Diaria, Semanal, Mensual, Anual

2. Lógica de auto-creación (hooks/useRecurrence.ts)

```
import { useMutation, useQueryClient } from '@tanstack/react-query';
import { supabase } from '@/lib/supabase';
import { addDays, addWeeks, addMonths, addYears } from 'date-fns';
import type { Payment, Recurrence } from '@/types/payment.types';

export function useRecurrence() {
  const queryClient = useQueryClient();

  const createNextRecurrence = useMutation({
    mutationFn: async (payment: Payment) => {
      if (payment.recurrence === 'none') return null;

      const currentDueDate = new Date(payment.due_date!);
      let nextDueDate: Date;

      switch (payment.recurrence) {
        case 'daily':
          nextDueDate = addDays(currentDueDate, 1);
          break;
        case 'weekly':
          nextDueDate = addWeeks(currentDueDate, 1);
          break;
        case 'monthly':
          nextDueDate = addMonths(currentDueDate, 1);
          break;
        case 'yearly':
          nextDueDate = addYears(currentDueDate, 1);
          break;
        default:
          return null;
      }

      // Crear nuevo pago con mismos datos pero nueva fecha
      const { data, error } = await supabase
        .from('payments')
        .insert([
          {
            name: payment.name,
            amount: payment.amount,
            category: payment.category,
            responsible: payment.responsible,
            due_date: nextDueDate.toISOString().split('T')[0],
            notes: payment.notes,
            icon: payment.icon,
            color: payment.color,
            recurrence: payment.recurrence,
            parent_payment_id: payment.id,
            user_id: payment.user_id,
            is_paid: false,
          }
        ])
        .select()
        .single();

      if (error) throw error;
      return data;
    },
  });
}
```

```
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ['payments'] });
      },
    });
    return { createNextRecurrence };
  }
}
```

3. Modificar togglePaid para crear siguiente

- Al marcar como pagado, verificar si es recurrente
- Si es recurrente, llamar createNextRecurrence

4. Vista de Series de Pagos

- Ver historial de un pago recurrente
- Pausar/reanudar recurrencia
- Editar toda la serie o solo una instancia

■ Entregable Fase 5:

- Pagos recurrentes funcionando
- Auto-creación de siguientes pagos
- Gestión de series

FASE 6: Dashboard & Analytics (2-3 días)

Objetivos:

- Vista general de estado financiero
- Estadísticas y gráficos
- KPIs principales

Componentes del Dashboard:

1. Cards de Resumen (StatsCard.tsx)

- Total pendiente este mes
- Pagos vencidos (con alerta roja)
- Próximos 7 días
- Total pagado este mes

2. Gráfico de Categorías (CategoryChart.tsx)

- Pie chart o bar chart
- Gastos por categoría del mes actual
- Usar recharts o similar

3. Gráfico de Tendencia Mensual

- Line chart con últimos 6 meses
- Comparar gasto mensual

4. Top Responsables

- Ranking de responsables con más pagos pendientes
- Total por responsable

5. Próximos Vencimientos (widget)

- Lista de próximos 5 pagos a vencer
- Con countdown visual

```
// Ejemplo con recharts
import { PieChart, Pie, Cell, ResponsiveContainer, Legend, Tooltip } from 'recharts';
import { CATEGORIES } from '@config/categories';

export function CategoryChart({ payments }: { payments: Payment[] }) {
  // Agrupar por categoría
  const data = Object.entries(CATEGORIES).map(([key, config]) => {
    const total = payments
      .filter(p => p.category === key && !p.is_paid)
      .reduce((sum, p) => sum + (p.amount || 0), 0);

    return {
      name: config.label,
      value: total,
      color: config.color,
    };
  });

  return (
    <ResponsiveContainer width="100%" height={300}>
      <PieChart>
        <Pie
          data={data}
          data-bbox="125 665 797 923"/>
```

```

      dataKey="value"
      nameKey="name"
      cx="50%"
      cy="50%"
      outerRadius={80}
      label
    >
      {data.map((entry, index) => (
        <Cell key={index} fill={entry.color} />
      ))}
    </Pie>
    <Tooltip formatter={(value) => `$$${value}`} />
    <Legend />
  </PieChart>
</ResponsiveContainer>
);
}

```

■ Entregable Fase 6:

- Dashboard funcional con KPIs
- Gráficos de categorías y tendencias
- Widgets de próximos vencimientos

FASE 7: Integración & Deploy (1-2 días)

Objetivos:

- Preparar módulo para integración
- Documentación de uso
- Deploy standalone o como submodule

Tareas:

1. Crear componente principal exportable (src/index.tsx)

```
// src/index.tsx
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { PaymentsDashboard } from '../components/Dashboard/PaymentsDashboard';
import { PaymentsList } from '../components/PaymentsList/PaymentsList';

const queryClient = new QueryClient();

export interface PaymentsModuleProps {
  userId?: string;
  defaultResponsible?: string;
  view?: 'dashboard' | 'list' | 'kanban' | 'calendar';
  onPaymentCreated?: (payment: Payment) => void;
  onPaymentUpdated?: (payment: Payment) => void;
  onPaymentDeleted?: (id: string) => void;
}

export function PaymentsModule({
  userId,
  defaultResponsible,
  view = 'dashboard',
  onPaymentCreated,
  onPaymentUpdated,
  onPaymentDeleted,
}: PaymentsModuleProps) {
  return (
    <QueryClientProvider client={queryClient}>
      <div className="payments-module">
        {view === 'dashboard' && <PaymentsDashboard />}
        {view === 'list' && <PaymentsList />}
        {/* ... otras vistas */}
      </div>
    </QueryClientProvider>
  );
}

// También exportar componentes individuales por si quieren usarlos separados
export { PaymentCard } from '../components/PaymentCard/PaymentCard';
export { CategoryBadge } from '../components/shared/CategoryBadge';
export { usePayments } from '../hooks/usePayments';
export type { Payment, Category, Recurrence } from '../types/payment.types';
```

2. Configurar como Git Submodule

```
# En tu proyecto principal (ej: sistema-inmobiliario)
git submodule add https://github.com/abrahamkohan/payments-reminders.git src/modules/payments

# Para usar el módulo:
# 1. Copiar .env.example a .env en el módulo
# 2. Configurar variables de Supabase
# 3. Importar en tu app:

import { PaymentsModule } from '@modules/payments';

function App() {
  return (
    <div>
      <PaymentsModule
        userId={user.id}
        defaultResponsible={user.name}
      />
    </div>
  );
}
```

```
        view="dashboard"
        onPaymentCreated={({payment}) => {
          console.log('Pago creado:', payment);
        }}
      />
    </div>
  );
}
```

3. Crear README.md completo

- Instrucciones de instalación
- Variables de entorno necesarias
- Ejemplos de uso
- API del módulo

4. Deploy standalone (opcional)

- Si querés usarlo como app independiente
- Deploy en Vercel o Netlify
- Agregar auth con Supabase Auth

■ Entregable Fase 7:

- Módulo listo para integrar
- Documentación completa
- Deploy funcional (si aplica)

5. ■ MEJORAS SOBRE NOTION

Este módulo incluye funcionalidades que Notion no tiene de forma nativa:

Funcionalidad	Notion	Módulo Personalizado
Recordatorios automáticos	■ No	■ Emails automáticos
Pagos recurrentes	■ Manual	■ Auto-creación
Dashboard analytics	■ No	■ KPIs + gráficos
Vista calendario	■ Timeline básico	■ Calendario interactivo
Vista kanban	■ Sí	■ Sí + drag & drop
Marcar como pagado	■ Checkbox	■ Checkbox + timestamp
Iconos custom por ítem	■ Solo por DB	■ Por cada pago
Colores personalizados	■ Solo categoría	■ Override por pago
Historial de pagados	■ Difícil filtrar	■ Vista dedicada
Export datos	■ CSV	■ CSV/Excel/PDF
Búsqueda avanzada	■ Básica	■ Múltiples filtros
Mobile-first	■ Desktop-first	■ Responsive nativo

6. ■ GUÍA UI/UX MINIMALISTA

Paleta de Colores

- **Fondo general:** bg-gray-50 (#f9fafb)
- **Cards:** bg-white con shadow-sm, hover:shadow-md
- **Bordes:** border-gray-200
- **Texto primario:** text-gray-900
- **Texto secundario:** text-gray-600
- **Acentos:** Según categoría (azul, verde, naranja, etc.)

Categorías con Iconos y Colores

Categoría	Emoji	Color	Tailwind Classes
Servicios	■	Azul	bg-blue-100 text-blue-700
Expensas	■	Verde	bg-green-100 text-green-700
Honorarios	■	Naranja	bg-orange-100 text-orange-700
Impuestos	■	Púrpura	bg-purple-100 text-purple-700
Trámites	■	Amarillo	bg-yellow-100 text-yellow-700
Otros	■	Gris	bg-gray-100 text-gray-700

Componentes Clave

1. PaymentCard

```
<div className="bg-white rounded-lg shadow-sm hover:shadow-md transition-shadow p-4 border border-gray-200">
  <div className="flex items-start justify-between">
    <div className="flex items-center gap-3">
      <span className="text-2xl">{payment.icon}</span>
      <div>
        <h3 className="font-semibold text-gray-900">{payment.name}</h3>
        <p className="text-sm text-gray-600">{payment.responsible}</p>
      </div>
    </div>
    <div className="text-right">
      <p className="font-bold text-lg text-gray-900">
        ${payment.amount}
      </p>
      <DueDateBadge date={payment.due_date} />
    </div>
  </div>
  <div className="mt-3 flex items-center gap-2">
    <CategoryBadge category={payment.category} />
    <Checkbox checked={payment.is_paid} onChange={handleToggle} />
  </div>
</div>
```

2. CategoryBadge

```
<span className={cn(
  "inline-flex items-center gap-1.5 px-2.5 py-1 rounded-full text-xs font-medium",
```



```

    CATEGORIES[category].bgClass,
    CATEGORIES[category].textClass
  })>
  <span>{CATEGORIES[category].icon}</span>
  <span>{CATEGORIES[category].label}</span>
</span>

```

3. DueDateBadge (con lógica de colores)

```

function DueDateBadge({ date }: { date: string }) {
  const daysUntil = differenceInDays(new Date(date), new Date());

  const badgeClasses = cn(
    "px-2 py-1 rounded text-xs font-medium",
    {
      "bg-red-100 text-red-700": daysUntil < 0, // Vencido
      "bg-orange-100 text-orange-700": daysUntil >= 0 && daysUntil <= 3,
      "bg-yellow-100 text-yellow-700": daysUntil > 3 && daysUntil <= 7,
      "bg-gray-100 text-gray-700": daysUntil > 7,
    }
  );

  return (
    <span className={badgeClasses}>
      {format(new Date(date), 'dd/MM/yyyy')}
    </span>
  );
}

```

Tipografía

- Font: Inter o System UI
- Títulos: font-bold
- Cuerpo: font-normal
- Tamaños: text-sm (12px), text-base (16px), text-lg (18px)

Espaciado y Rounding

- Padding cards: p-4 o p-6
- Gaps: gap-2, gap-3, gap-4
- Border radius: rounded-lg (8px)
- Shadows: shadow-sm, hover:shadow-md

7. ■ INTEGRACIÓN CON CRM EXISTENTE

Pasos para integrar el módulo en tu sistema-inmobiliario existente:

Paso 1: Agregar como Git Submodule

```
# En la raíz de sistema-inmobiliario
cd sistema-inmobiliario
git submodule add https://github.com/abrahamkohan/payments-reminders.git src/modules/payments

# Inicializar el submodule
git submodule init
git submodule update
```

Paso 2: Instalar dependencias del módulo

```
cd src/modules/payments npm install # 0 desde la raíz: npm install --prefix src/modules/payments
```

Paso 3: Configurar variables de entorno

```
# En sistema-inmobiliario/.env (agregar estas variables)
VITE_SUPABASE_URL=tu_url_de_supabase
VITE_SUPABASE_ANON_KEY=tu_anon_key
VITE_RESEND_API_KEY=tu_resend_key # Para emails
```

Paso 4: Importar y usar en tu CRM

```
// En tu CRM, ej: src/pages/Pagos.tsx
import { PaymentsModule } from '@modules/payments';
import { useAuth } from '@hooks/useAuth';

export function PagosPage() {
  const { user } = useAuth();

  return (
    <div className="container mx-auto p-6">
      <h1 className="text-2xl font-bold mb-6">Gestión de Pagos</h1>

      <PaymentsModule
        userId={user.id}
        defaultResponsible={user.name}
        view="dashboard"
        onPaymentCreated={(payment) => {
          // Opcional: integrar con otras partes del CRM
          console.log('Nuevo pago creado:', payment);
        }}
      />
    </div>
  );
}
```

Paso 5: Agregar ruta en React Router

```
// En tu router principal
import { PagosPage } from '@pages/Pagos';

const router = createBrowserRouter([
```

```
// ... tus rutas existentes
{
  path: '/pagos',
  element: <PagosPage />,
},
]);
```

Paso 6: Agregar al menú de navegación

```
// En tu Sidebar o Navigation component
<nav>
  <Link to="/dashboard">Dashboard</Link>
  <Link to="/clientes">Clientes</Link>
  <Link to="/propiedades">Propiedades</Link>
  <Link to="/pagos">■ Pagos</Link> { /* Nueva entrada */ }
</nav>
```

■■ **Nota importante:** El módulo usa su propia instancia de QueryClient. Si tu CRM ya usa React Query, podés compartir el QueryClient pasándolo como prop.

Compartir QueryClient (opcional):

```
// En tu App.tsx principal import { QueryClient, QueryClientProvider } from
'@tanstack/react-query'; import { PaymentsModule } from '@modules/payments'; const queryClient =
new QueryClient(); function App() { return ( { /* Tu app */ } { /* Pasar el mismo client */ } ); }
```

8. ■ USO STANDALONE (INDEPENDIENTE)

Si querés usar el módulo como aplicación independiente (sin integrarlo a otro proyecto):

Paso 1: Clonar el repositorio

```
git clone https://github.com/abrahamkohan/payments-reminders.git
cd payments-reminders
npm install
```

Paso 2: Configurar variables de entorno

```
# Copiar .env.example a .env cp .env.example .env # Editar .env con tus credenciales de Supabase
VITE_SUPABASE_URL=https://tu-proyecto.supabase.co VITE_SUPABASE_ANON_KEY=tu_anon_key_aqui
VITE_RESEND_API_KEY=tu_resend_key
```

Paso 3: Crear las tablas en Supabase

- Ir a Supabase SQL Editor
- Ejecutar el SQL del schema (ver Fase 1)
- Habilitar Row Level Security (RLS)

Paso 4: Agregar autenticación (si querés multi-usuario)

```
// En src/App.tsx
import { Auth } from '@supabase/auth-ui-react';
import { ThemeSupa } from '@supabase/auth-ui-shared';
import { supabase } from '../lib/supabase';
import { PaymentsModule } from './index';

function App() {
  const [session, setSession] = useState(null);

  useEffect(() => {
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
    });

    const {
      data: { subscription },
    } = supabase.auth.onAuthStateChange((_event, session) => {
      setSession(session);
    });

    return () => subscription.unsubscribe();
  }, []);

  if (!session) {
    return (
      <div className="container mx-auto max-w-md p-8">
        <h1 className="text-2xl font-bold mb-4">Iniciar Sesión</h1>
        <Auth
          supabaseClient={supabase}
          appearance={{ theme: ThemeSupa }}
          providers={['google']}
        />
      </div>
    );
  }
}
```

```
}  
    return <PaymentsModule userId={session.user.id} />;  
}
```

Paso 5: Ejecutar en desarrollo

```
npm run dev  
  
# Abrir http://localhost:5173
```

Paso 6: Deploy a producción

```
# Build  
npm run build  
  
# Deploy en Vercel  
vercel deploy  
  
# O deploy en Netlify  
netlify deploy --prod  
  
# Variables de entorno en Vercel/Netlify:  
# - VITE_SUPABASE_URL  
# - VITE_SUPABASE_ANON_KEY  
# - VITE_RESEND_API_KEY
```

■ **Ventaja del enfoque standalone: Tenés una app completamente funcional que podés compartir con clientes o usar internamente sin necesidad de integrarla a otro sistema.**

9. ■■ ESTIMACIÓN DE TIEMPOS

Fase	Tareas Principales	Días
1. Setup & Arquitectura	Proyecto, tipos, Supabase, migraciones	1-2
2. Componentes & CRUD	Cards, formularios, hooks, queries	2-3
3. Vistas y Filtros	Tabla, Kanban, Calendario, filtros	2-3
4. Recordatorios	Edge function, emails, cron job	2-3
5. Recurrencia	Auto-creación, gestión series	2
6. Dashboard	KPIs, gráficos, analytics	2-3
7. Integración	Export, docs, deploy	1-2
	TOTAL DESARROLLO	12-18 días
	Testing & refinamiento	3-5
	TOTAL PROYECTO	15-23 días

Estimación por velocidad:

- **Full-time (8h/día):** 3-4 semanas
- **Part-time (4h/día):** 6-8 semanas
- **Weekend warrior (2h/día):** 2-3 meses

■ **Recomendación:** Empezar con Fases 1-3 para tener MVP funcional en ~1 semana, luego iterar agregando recordatorios y recurrencia.

10. ■ PROMPT PARA CLAUDE CODE

Copí y pegá este prompt directamente en Claude Code para que construya el módulo completo:

Necesito que me construyas un módulo completo de gestión de Pagos y Recordatorios usando React 19, Vite, TypeScript, Tailwind CSS 4, Supabase y React Query. El módulo debe ser modular y reutilizable (como Git submodule) para poder integrarlo en otros proyectos React, pero también debe poder funcionar standalone. ## ESTRUCTURA DE DATOS Base de datos en Supabase (PostgreSQL) con estas tablas: **payments:** - id (UUID, PK) - created_at, updated_at (timestamps) - name (text, requerido) - amount (decimal) - category (enum: servicios, expensas, honorarios, impuestos, tramites, otros) - responsible (text) - due_date (date) - notes (text) - icon (text, emoji) - color (text, hex color) - is_paid (boolean, default false) - paid_at (timestamp) - recurrence (enum: none, daily, weekly, monthly, yearly) - parent_payment_id (UUID, FK to payments) - user_id (UUID, FK to auth.users) **payment_reminders:** - id (UUID, PK) - payment_id (UUID, FK) - reminder_date (timestamp) - days_before (integer) - is_sent (boolean) - sent_at (timestamp) - channel (enum: email, push, sms) Incluir índices, RLS policies y triggers para updated_at. ## FUNCIONALIDADES PRINCIPALES #### FASE 1: Base (PRIORIDAD ALTA) 1. Setup completo del proyecto (Vite + React 19 + TS + Tailwind 4) 2. Configuración Supabase client 3. Tipos TypeScript completos 4. Migraciones SQL para crear tablas 5. Config de categorías con iconos y colores: - Servicios: ■ azul - Expensas: ■ verde - Honorarios: ■ naranja - Impuestos: ■ púrpura - Trámites: ■ amarillo - Otros: ■ gris #### FASE 2: CRUD & Componentes 1. Hook usePayments con React Query (queries + mutations) 2. PaymentCard component (card minimalista con hover) 3. PaymentForm (crear/editar con React Hook Form + Zod) 4. CategoryBadge (badge con icono + color) 5. DueDateBadge (colores según proximidad: rojo=vencido, naranja=1-3 días, amarillo=4-7 días, gris=>7 días) 6. EmojiPicker component 7. ColorPicker component 8. CRUD completo: crear, editar, eliminar, duplicar, marcar pagado #### FASE 3: Vistas y Filtros 1. Vista Tabla (TableView) - similar a Notion 2. Vista Kanban (KanbanView) - columnas por categoría con drag & drop 3. Vista Calendario (CalendarView) - mensual con pagos por día 4. Sistema de filtros: - Por categoría (multi-select) - Por responsable (multi-select) - Por estado (todos/pendientes/pagados) - Por rango de fechas - Búsqueda texto 5. Ordenamiento por: fecha, monto, nombre, creación #### FASE 4: Recordatorios (OPCIONAL PRIMERO) 1. Edge Function en Supabase que corre diariamente 2. Lógica para crear recordatorios 7, 3 y 1 día antes 3. Envío de emails con Resend 4. Configuración cron job con pg_cron #### FASE 5: Recurrencia (OPCIONAL PRIMERO) 1. Campo recurrencia en formulario 2. Auto-creación del siguiente pago al marcar como pagado 3. Vista historial de serie de pagos 4. Pausar/editar recurrencia #### FASE 6: Dashboard 1. KPI cards: - Total pendiente mes actual - Pagos vencidos (alerta roja) - Próximos 7 días - Total pagado mes 2. Gráfico pie/bar de gastos por categoría (recharts) 3. Gráfico línea tendencia mensual (últimos 6 meses) 4. Top responsables 5. Widget próximos vencimientos #### FASE 7: Integración 1. Componente principal exportable PaymentsModule con props: - userId, defaultResponsible, view, callbacks 2. README completo con instrucciones de uso 3. .env.example 4. package.json configurado para submodule ## DISEÑO UI/UX **Estilo minimalista:** - Paleta: bg-gray-50, cards bg-white con shadow-sm/hover:shadow-md - Bordes: border-gray-200 - Font: Inter o system-ui - Spacing: p-4/p-6, gap-2/3/4 - Rounding: rounded-lg - Colores acentos según categoría **Componentes visuales clave:** - Cards con checkbox para marcar pagado - Badges de categoría con icono + color - Badges de fecha con lógica de colores - Forms con validación visual - Emoji picker integrado - Transiciones suaves ## PRIORIDAD DE DESARROLLO 1. **MVP (Fase 1-2):** Setup + CRUD básico → ~2-3 días 2. **Versión completa UI (+ Fase 3):** Vistas + filtros → +2-3 días 3. **Features avanzados (Fases 4-5):** Recordatorios + recurrencia → +3-4 días 4. **Polish (Fases 6-7):** Dashboard + integración → +2-3 días Empezá por el MVP (Fases 1-2) para tener algo funcional rápido. ## ESTRUCTURA DE CARPETAS ``` payments-reminders/ ├── src/ ├── components/ ├── PaymentsList/ ├── PaymentForm/ ├── PaymentViews/ ├── Dashboard/ ├── shared/ ├── hooks/ ├── lib/ ├── types/ ├── config/ ├── index.tsx ├── supabase/ ├── migrations/ ├── functions/ ├── package.json ├── README.md ``` ## REQUISITOS TÉCNICOS - Usar React 19 features (use, useMemo optimizado) - TypeScript estricto (no any) - Tailwind 4 con CSS variables - React Query para todo el estado del servidor - React Hook Form + Zod para formularios - date-fns para manejo de fechas - Supabase client configurado con tipos generados - Mobile-first responsive - Accesibilidad (ARIA labels, keyboard navigation) Comenzá creando la estructura base, las migraciones SQL y el hook usePayments. Luego seguí con los componentes visuales. Preguntame si tenés dudas sobre alguna decisión de arquitectura. Quiero código production-ready, bien comentado, con manejo de errores y loading states.

■ Instrucciones de uso:

1. Abrí Claude Code en tu terminal
2. Navegá a la carpeta donde querés crear el proyecto
3. Copiá y pegá el prompt completo
4. Claude Code te guiará paso a paso en la construcción
5. Revisá el código generado y hacé ajustes según necesites



■ Módulo de Pagos y Recordatorios - Roadmap Completo

Generado el 14/04/2026

Abraham Kohan - PY Advisors

